PRACTICAL LIST

ITUS102: Web Technology Framework

Semester: 1st | Total Labs: 15 | Total Practical Hours: 30

Practical No.	Problem Definition	CO/PO
1	**Aim**: To understand the fundamentals of web architecture by designing a semantic HTML5 webpage using appropriate structural elements and implementing version control using Git and GitHub for effective project management and collaboration. **Industry Scenario**: GreenLeaf Organics Pvt. Ltd. is a newly established company specializing in organic food products. To establish its online presence and improve customer outreach, the company has decided to launch its official website. As a Front-End Web Developer, you have been assigned the responsibility of developing the first version of the company's homepage. The management expects the website to follow HTML5 semantic standards so that it is well-structured, accessible, maintainable, and search engine friendly. The homepage should present the company's profile, products, services, featured offerings, and contact information using appropriate semantic HTML5 elements. Since the project will be enhanced by multiple developers in future development phases, the project manager has instructed that the project source code be managed using Git and hosted on GitHub to facilitate version control, collaboration, and project tracking. Based on the above requirements, design and develop the homepage using semantic HTML5 elements and maintain the project using Git and GitHub. **Tasks to be Performed** 1. Install and configure Git and create a GitHub account (if not already available). 2. Create a new project directory and initialize a local Git repository. 3. Develop a semantic HTML5 webpage using appropriate structural elements such as <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer>. 4. Design the homepage to include: • Company logo • Navigation menu • About the company • Products/Services • Featured section or image banner • Contact information • Footer 5. Use suitable HTML elements including headings, paragraphs, lists, hyperlinks, images, tables, and forms wherever applicable. 6. Add appropriate metadata including page title, character encoding, viewport settings, author information, description, and favicon. 7. Validate the HTML document using the W3C HTML Validator and resolve validation errors, if any. 8. Commit the project to the local Git repository using meaningful commit messages.	CO1

9. Create a GitHub repository and push the project to the remote repository.
10. Verify the successful upload of the project by checking the repository contents and commit history.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Compare semantic and non-semantic HTML elements. Under what circumstances would you still use a generic element instead of a semantic HTML5 element?
2. Suppose multiple developers are simultaneously modifying different sections of the same webpage. Explain how Git assists in managing version control and resolving code conflicts.
3. If a webpage is developed without semantic HTML5 elements, what challenges might arise in terms of accessibility, maintainability, and search engine indexing?

Supplementary Problems

1. Design a semantic HTML5 homepage for a college department containing sections such as About, Faculty, Academic Programs, Events, and Contact Information.
2. Develop a semantic webpage for a non-governmental organization (NGO) highlighting its mission, ongoing projects, volunteer opportunities, and donation details.
3. Create a project README.md file describing the project objective, folder structure, technologies used, and steps to execute the webpage locally.

Key Skills to be Addressed

- Understanding web architecture fundamentals.
- Creating semantic HTML5 webpages.
- Structuring web content using appropriate HTML5 elements.
- Developing accessible and standards-compliant webpages.
- Applying version control using Git.
- Managing source code repositories using GitHub.

Learning Outcome

Upon successful completion of this practical, students will be able to design semantic HTML5 webpages using appropriate structural elements, explain the importance of semantic markup in modern web development, and manage web development projects efficiently using Git and GitHub.

Software / Tools / Platforms to be Used

- Visual Studio Code
- HTML5
- Google Chrome / Mozilla Firefox
- Git
- GitHub
- W3C HTML Validator

Total Hours of Engagement

2 Hours

<table>
<tr>
<td></td>
<td>

1.5 Hours: Design and Implementation
30 Minutes: Testing, GitHub Submission, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)
<table>
<tr><th>Criteria</th><th>Weight (%)</th></tr>
<tr><td>Correct implementation of semantic HTML5 elements, webpage structure, navigation, multimedia components, and metadata following HTML5 standards.</td><td>40%</td></tr>
<tr><td>Ability to explain semantic HTML5 concepts, webpage organization, accessibility, and web architecture fundamentals.</td><td>25%</td></tr>
<tr><td>Effective use of Git and GitHub including repository creation, meaningful commit history, version management, and project synchronization.</td><td>20%</td></tr>
<tr><td>Code quality, readability, indentation, documentation, naming conventions, project organization, and adherence to web development best practices.</td><td>15%</td></tr>
</table>
</td>
<td></td>
</tr>
<tr>
<td>2</td>
<td>
Aim: To apply CSS3 concepts for designing responsive, visually appealing, and user-friendly web pages using Flexbox, CSS Grid, media queries, cascading, and inheritance principles.

Industry Scenario
CHARUSAT University is redesigning its Department of Computer Engineering website to provide a consistent browsing experience across desktops, tablets, and smartphones. The existing website has fixed layouts, inconsistent styling, and poor readability on smaller screens, resulting in reduced user engagement. As a Front-End UI Developer, you have been assigned the responsibility of redesigning the department homepage using modern CSS3 techniques. The university expects the website to maintain a professional appearance while automatically adapting to different screen sizes. The webpage should demonstrate effective use of selectors, cascading, inheritance, Flexbox, CSS Grid, and media queries to create a responsive and visually consistent layout.
Based on the above requirements, design and develop a responsive department homepage using CSS3 best practices.

Tasks to be Performed

Create a semantic HTML5 webpage representing the Department of Computer Engineering.
Apply CSS3 selectors to style different HTML elements, classes, and IDs.
Implement typography, colors, spacing, borders, backgrounds, and box model properties to enhance the visual appearance.
Demonstrate the concepts of cascading, inheritance, and CSS specificity while styling the webpage.
Design the navigation menu and page sections using CSS Flexbox.
Create a responsive content layout using CSS Grid.
Implement media queries to optimize the webpage for desktop, tablet, and mobile devices.
Ensure images, navigation menus, and content resize appropriately across different screen resolutions.

</td>
<td>CO2</td>
</tr>
</table>

9. Validate the CSS code and verify the responsiveness using browser developer tools.
10. Organize the project by separating HTML and CSS into appropriate files following standard naming conventions.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Compare Flexbox and CSS Grid. Which layout technique is more suitable for one-dimensional and two-dimensional webpage layouts? Justify your answer with examples from your implementation.
2. Explain how cascading, inheritance, and specificity influence the final appearance of a webpage when multiple CSS rules are applied.
3. Discuss the importance of media queries in responsive web design. What challenges would arise if responsive techniques were not implemented?
4. A webpage displays correctly on a desktop but appears distorted on a mobile device. Explain a systematic approach to identify and resolve the issue using browser developer tools.

Supplementary Problems

1. Design a responsive homepage for a hospital displaying departments, doctors, emergency services, and contact information.
2. Create a responsive landing page for an e-commerce website using Flexbox and CSS Grid to organize products and promotional banners.
3. Develop a responsive portfolio webpage for a software developer that adapts seamlessly across desktop, tablet, and mobile devices.

Key Skills to be Addressed

- Applying CSS3 styling techniques.
- Understanding selectors, cascading, inheritance, and specificity.
- Designing responsive layouts using Flexbox and CSS Grid.
- Implementing media queries for different screen sizes.
- Organizing and maintaining external stylesheet files.
- Debugging CSS using browser developer tools.

Learning Outcome

Upon successful completion of this practical, students will be able to design responsive and visually appealing webpages using CSS3 by effectively applying Flexbox, CSS Grid, media queries, cascading, inheritance, and responsive design principles.

Software / Tools / Platforms to be Used

- Visual Studio Code
- HTML5
- CSS3
- Google Chrome / Mozilla Firefox
- Browser Developer Tools

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design and Implementation

<table>
<tr>
<td></td>
<td>

0.5 Hour: Validation, Browser Testing, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)
<table>
<tr><th>Criteria</th><th>Weight (%)</th></tr>
<tr><td>Correct implementation of CSS3 styling, Flexbox, CSS Grid, media queries, cascading, and inheritance concepts.</td><td>40%</td></tr>
<tr><td>Ability to explain responsive web design principles, CSS layout techniques, and browser compatibility concepts.</td><td>25%</td></tr>
<tr><td>Quality of webpage responsiveness across multiple screen sizes, visual consistency, and effective use of browser developer tools.</td><td>20%</td></tr>
<tr><td>Code organization, readability, naming conventions, documentation, and adherence to CSS coding standards.</td><td>15%</td></tr>
</table>
</td>
<td></td>
</tr>
<tr>
<td>3</td>
<td>
Aim: To develop interactive JavaScript programs by applying variables, operators, conditional statements, loops, and functions for solving real-world problems.

Industry Scenario
Secure Bank Ltd. is developing an online banking portal to assist customers in performing basic financial calculations before applying for banking services. As a JavaScript Developer, you have been assigned the task of developing client-side modules that can perform operations such as loan eligibility checking, EMI estimation, interest calculation, and account-related utilities.
The management requires these modules to execute efficiently within the browser without requiring server-side processing. The application should demonstrate the use of JavaScript variables, operators, decision-making statements, loops, and user-defined functions to process user inputs and display meaningful results.
Based on the above requirements, develop JavaScript programs that implement the required banking utilities using appropriate programming constructs.

Tasks to be Performed

Create an HTML webpage and link an external JavaScript file.
Declare and initialize variables using var, let, and const, and demonstrate their scope.
Implement arithmetic, relational, logical, assignment, and conditional operators to perform banking-related calculations.
Develop programs using if, if-else, nested if, and switch statements to evaluate user-defined conditions.
Implement for, while, and do-while loops to perform repetitive calculations such as generating account summaries or interest tables.
Create reusable JavaScript functions with parameters and return values to perform financial computations.
Accept user input using appropriate browser methods or HTML input controls and display the computed results dynamically.
Organize the JavaScript code using proper indentation, comments, and naming conventions.
Test the application with different input values and verify the correctness of the output.

</td>
<td>CO3</td>
</tr>
</table>

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Compare the use of var, let, and const in JavaScript. Which declaration is most appropriate for modern web development? Justify your answer.
2. Differentiate between if-else statements and the switch statement. In which situations is each more suitable?
3. Compare the working of for, while, and do-while loops. Identify scenarios where each loop is most appropriate.

Supplementary Problems

1. Develop a Student Result Calculator that computes grades and classifies students based on their marks.
2. Create an Electricity Bill Calculator that determines the total bill amount using slab-wise tariff calculations.
3. Design a Simple Shopping Bill Generator that calculates the total amount, discount, GST, and final payable amount based on user inputs.

Key Skills to be Addressed

- Understanding JavaScript programming fundamentals.
- Working with variables, data types, and operators.
- Applying decision-making and iterative control structures.
- Developing reusable JavaScript functions.
- Accepting and processing user input.
- Debugging JavaScript programs using browser developer tools.

Learning Outcome

Upon successful completion of this practical, students will be able to develop JavaScript programs using variables, operators, conditional statements, loops, and functions to solve real-world computational problems and implement interactive client-side functionality.

Software / Tools / Platforms to be Used

- Visual Studio Code
- HTML5
- JavaScript (ES6)
- Google Chrome / Mozilla Firefox
- Browser Developer Tools

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design and Implementation
- **0.5 Hour:** Program Testing, Debugging, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of JavaScript variables, operators, conditional statements, loops, and functions to solve the given problem.	40%

	Ability to explain JavaScript programming concepts, control structures, and function implementation. **25%** Logical correctness of the solution, proper handling of user inputs, testing with different cases, and debugging. **20%** Code organization, readability, naming conventions, comments, and adherence to JavaScript coding standards. **15%**	
4	**Aim:** To develop interactive web pages by manipulating HTML elements using the Document Object Model (DOM) and implementing event handling techniques with JavaScript. **Industry Scenario** ShopEase Online Store is upgrading its e-commerce website to provide customers with a more interactive shopping experience. The current website displays static content, making it difficult for users to interact with products and navigate efficiently. As a Front-End JavaScript Developer, you have been assigned the responsibility of enhancing the user interface by implementing dynamic features using JavaScript. The management expects the application to respond to user actions such as button clicks, mouse events, keyboard inputs, and form interactions without reloading the webpage. The solution should dynamically update webpage content, modify styles, create and remove elements, and improve the overall user experience through effective use of the Document Object Model (DOM) and event handling. Based on the above requirements, develop an interactive web page using DOM manipulation and JavaScript event handling techniques. **Tasks to be Performed** 1. Create a webpage containing appropriate HTML elements such as headings, buttons, images, forms, lists, and input fields. 2. Access and manipulate HTML elements using DOM methods such as getElementById(), getElementsByClassName(), getElementsByTagName(), querySelector(), and querySelectorAll(). 3. Modify the content, attributes, and CSS styles of HTML elements dynamically using JavaScript. 4. Create, append, replace, and remove HTML elements dynamically using DOM manipulation methods. 5. Implement event handling for user interactions such as click, double-click, mouseover, mouseout, keydown, keyup, and change events. 6. Display appropriate feedback messages based on user actions using JavaScript. 7. Implement simple interactive features such as theme switching, image preview, dynamic menu, or live content updates. **Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation** 1. Explain the role of the Document Object Model (DOM) in creating dynamic and interactive web applications. 2. Compare inline, internal, and external event handling techniques. Which approach is considered the best practice? Justify your answer.	CO3

3. Differentiate between modifying an existing DOM element and creating a new DOM element dynamically. Provide suitable examples from your implementation.
4. How does event-driven programming improve the responsiveness and usability of modern web applications? Explain with suitable examples.

Supplementary Problems

1. Develop a Student Attendance Dashboard where attendance status updates dynamically based on user actions.
2. Design an Online Movie Ticket Booking interface that dynamically updates seat selection and booking summary using DOM manipulation.
3. Create an Interactive To-Do List that allows users to add, edit, mark as completed, and delete tasks dynamically.

Key Skills to be Addressed

- Understanding the Document Object Model (DOM).
- Selecting and manipulating HTML elements dynamically.
- Handling browser events using JavaScript.
- Creating responsive and interactive user interfaces.
- Dynamically modifying webpage content and styles.
- Debugging JavaScript applications using browser developer tools**.**

Learning Outcome

Upon successful completion of this practical, students will be able to develop interactive web applications by manipulating HTML elements through the Document Object Model (DOM) and implementing event-driven programming techniques using JavaScript**.**

Learning Outcome

Upon successful completion of this practical, students will be able to develop interactive web applications by manipulating HTML elements through the Document Object Model (DOM) and implementing event-driven programming techniques using JavaScript.

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design and Implementation
- **0.5 Hour:** Testing, Debugging, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	**Weight (%)**
Correct implementation of DOM manipulation techniques, dynamic content updates, and event handling mechanisms.	**40%**
Ability to explain DOM concepts, event-driven programming, and JavaScript interaction with HTML elements.	**25%**
Functionality of interactive features, debugging approach, and proper handling of user events.	**20%**
Code organization, readability, naming conventions, modularity, and adherence to JavaScript coding standards.	**15%**

5	**Aim:** To design interactive HTML forms and implement client-side validation using JavaScript, regular expressions, and secure input validation techniques to ensure accurate and reliable user data entry. **Industry Scenario** CityCare Hospital is launching an online patient registration portal to simplify appointment booking and reduce manual paperwork. As a Front-End Web Developer, you have been assigned the responsibility of developing a secure and user-friendly registration form for new patients. The hospital management requires that all user inputs be validated on the client side before submission to minimize incorrect or incomplete information. The registration form should validate mandatory fields, email addresses, mobile numbers, passwords, dates, and other user inputs using JavaScript and Regular Expressions (RegEx). Appropriate validation messages should guide users to correct invalid entries while ensuring that only valid data is accepted. Based on the above requirements, design and implement a secure patient registration form using HTML5 and JavaScript validation techniques. **Tasks to be Performed** 1. Create an HTML5 patient registration form containing appropriate input controls such as text fields, radio buttons, checkboxes, drop-down lists, date picker, email, password, and file upload. 2. Apply HTML5 form attributes such as required, placeholder, maxlength, min, max, and appropriate input types wherever applicable. 3. Implement JavaScript validation for mandatory fields before form submission. 4. Validate email addresses, mobile numbers, passwords, and names using Regular Expressions (RegEx). 5. Display meaningful validation messages for invalid or incomplete user inputs. 6. Prevent form submission until all validation rules are satisfied. 7. Implement password confirmation and password strength validation. 8. Apply secure input validation practices by trimming unnecessary spaces and restricting invalid characters. 9. Test the form using valid, invalid, boundary, and empty input values to verify proper validation. **Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation** 1. Compare HTML5 built-in validation and JavaScript-based validation. Under what circumstances should each approach be preferred? 2. Explain the importance of Regular Expressions (RegEx) in validating user inputs. Illustrate with suitable examples from your implementation. 3. Discuss the security risks associated with accepting unvalidated user input in web applications. How does client-side validation improve user experience while complementing server-side validation? **Supplementary Problems** 1. Develop an Online Student Admission Form with appropriate validation for academic and personal information.	CO3, CO5

2. Design an Employee Registration Portal that validates employee ID, email address, mobile number, salary, and password before submission.
3. Create an Online Event Registration Form with client-side validation for participant details, payment information, and confirmation messages.

Key Skills to be Addressed

- Designing user-friendly HTML5 forms.
- Applying HTML5 form validation attributes.
- Implementing JavaScript-based client-side validation.
- Using Regular Expressions (RegEx) for input validation.
- Displaying meaningful validation messages.
- Applying secure input validation techniques.

Learning Outcome

Upon successful completion of this practical, students will be able to design secure and user-friendly HTML forms, implement client-side validation using JavaScript and Regular Expressions, and ensure accurate data entry by applying appropriate validation techniques.

Software / Tools / Platforms to be Used

- Visual Studio Code
- HTML5
- CSS3
- JavaScript (ES6)
- Google Chrome / Mozilla Firefox
- Browser Developer Tools

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design and Implementation
- **0.5 Hour:** Validation Testing and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct design and implementation of HTML5 forms, JavaScript validation, Regular Expressions, and secure input validation techniques.	**40%**
Ability to explain form validation concepts, Regular Expressions, validation logic, and secure input handling practices.	**25%**
Correct handling of valid, invalid, boundary, and error input conditions with meaningful validation messages.	**20%**
Code organization, readability, naming conventions, documentation, and adherence to web development best practices.	**15%**

6	**Aim**: To develop asynchronous web applications using JavaScript timers, callbacks, Fetch API/AJAX, and browser storage techniques for efficient data retrieval and improved user experience.	CO3

Industry Scenario

WanderGo Travels Pvt. Ltd. is developing an online travel portal that allows users to explore travel destinations, view tour packages, and personalize their browsing experience. As a Front-End Web Developer, you have been assigned to enhance the website by implementing asynchronous features that improve responsiveness and reduce unnecessary page reloads.

The management requires the application to retrieve destination information from an external JSON source using the Fetch API, display promotional messages using timers, execute asynchronous operations through callbacks, and store user preferences such as selected destinations and themes using browser storage. The website should provide a smooth and interactive experience while efficiently handling user interactions.

Based on the above requirements, develop a JavaScript application demonstrating asynchronous programming concepts using timers, callbacks, Fetch API/AJAX, and browser storage.

Tasks to be Performed

1. Create a responsive webpage for the travel portal using HTML5 and CSS3.
2. Implement JavaScript timers (setTimeout() and setInterval()) to display promotional offers or update content dynamically.
3. Develop callback functions to execute asynchronous tasks in the required sequence.
4. Retrieve travel package information from a local JSON file or a public REST API using the Fetch API.
5. Process and display the retrieved JSON data dynamically on the webpage.
6. Handle loading states and errors using appropriate JavaScript techniques such as try...catch or .catch().
7. Store user preferences (e.g., selected destination or preferred theme) using Local Storage and retrieve them when the webpage reloads.
8. Use Session Storage to temporarily maintain user-specific information during the active browser session.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between synchronous and asynchronous execution in JavaScript. Why is asynchronous programming important for modern web applications?
2. Compare the working of callbacks, Fetch API, and AJAX. How do they improve user experience compared to traditional page reloads?
3. Explain the differences between Local Storage and Session Storage. In which situations would each be more appropriate?
4. While retrieving data from an external API, the network connection is interrupted. Explain how your application should handle such failures gracefully to maintain usability.

Supplementary Problems

1. Develop a Weather Information Dashboard that retrieves current weather details from a public REST API and stores the user's preferred city using Local Storage.

2. Create a News Headlines Viewer that fetches news articles from a JSON source and automatically refreshes the content at regular intervals using JavaScript timers.
3. Design a Movie Explorer application that retrieves movie information from a public API, remembers the user's preferred genre using browser storage, and displays results dynamically.

Key Skills to be Addressed

- Understanding asynchronous programming concepts.
- Implementing JavaScript timers and callback functions.
- Retrieving and processing JSON data using the Fetch API.
- Handling asynchronous errors effectively.
- Working with Local Storage and Session Storage.
- Dynamically updating webpage content.
- Developing responsive and interactive web applications

Learning Outcome

Upon successful completion of this practical, students will be able to implement asynchronous JavaScript applications using timers, callbacks, Fetch API, and browser storage to retrieve, process, and manage data efficiently while enhancing user interaction.

Software / Tools / Platforms to be Used

- Visual Studio Code
- HTML5
- CSS3
- JavaScript (ES6)
- Google Chrome / Mozilla Firefox
- Browser Developer Tools
- Public REST API or Local JSON File

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design and Implementation
- **0.5 Hour:** Testing, Debugging, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of timers, callbacks, Fetch API/AJAX, browser storage, and asynchronous data processing.	**40%**
Ability to explain asynchronous programming concepts, JSON handling, Fetch API, and browser storage mechanisms.	**25%**
Proper handling of API responses, error conditions, dynamic content updates, and storage management.	**20%**
Code organization, readability, modularity, documentation, and adherence to JavaScript coding standards.	**15%**

7	**Aim:** To develop a basic server-side web application using Node.js and Express.js that demonstrates request-response handling, routing, session management, and user authentication concepts.	CO5

Industry Scenario

Central City Library is digitizing its membership services by introducing an online portal for students and staff. The portal should allow registered members to log in securely and access library resources based on their authentication status. As a Full-Stack Web Developer, you have been assigned to develop the initial server-side application that manages user requests and controls access to protected resources.

The library management requires the application to demonstrate request-response handling, multiple routes, session-based user authentication, and access control. Authenticated users should be able to access protected pages, while unauthenticated users should be redirected to the login page. The application should also provide a logout facility to terminate the active session securely.

Based on the above requirements, develop a server-side application demonstrating routing, session management, and authentication using modern web development practices.

Tasks to be Performed

1. Install and configure Node.js and initialize a new Express.js project.
2. Create an Express server and configure the application to listen on an appropriate port.
3. Implement multiple routes to serve different pages such as Home, About, Login, Dashboard, and Logout.
4. Demonstrate request-response handling using appropriate HTTP request and response objects.
5. Configure session management using Express middleware.
6. Implement a simple login mechanism using predefined user credentials.
7. Restrict access to the Dashboard page so that only authenticated users can access it.
8. Redirect unauthenticated users attempting to access protected resources to the Login page.
9. Implement a Logout route that destroys the active session and redirects the user appropriately.
10. Test the application by verifying routing, authentication, session persistence, and logout functionality.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Explain the role of the request and response objects in the client-server communication process.
2. Differentiate between client-side routing and server-side routing with suitable examples.
3. Discuss the purpose of session management in web applications. How does it differ from browser storage mechanisms such as Local Storage and Session Storage?
4. Explain why authentication alone is insufficient without proper authorization and session management in secure web applications.

Supplementary Problems

1. Develop an Online Course Portal where authenticated students can access course materials while unauthorized users are redirected to the login page.
2. Create an Employee Attendance Portal with login, dashboard, profile, and logout functionalities using session management.
3. Design a Hotel Reservation Portal demonstrating user login, protected booking page access, and secure logout functionality.

Key Skills to be Addressed

- Understanding client-server architecture.
- Developing server-side applications using Node.js and Express.js.
- Implementing routing and request-response handling.
- Configuring session management.
- Implementing basic user authentication.

Learning Outcome

Upon successful completion of this practical, students will be able to develop a basic server-side web application that handles client requests, implements routing, manages user sessions, and demonstrates authentication concepts using Node.js and Express.js.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Node.js
- Express.js
- JavaScript (ES6)
- Google Chrome / Mozilla Firefox
- Postman (for testing HTTP requests)
- Browser Developer Tools

Total Hours of Engagement

2 Hours

- **1.5 Hours:** Design and Implementation
- **0.5 Hour:** Testing, Debugging, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of Express server, routing, request-response handling, session management, and authentication.	40%
Ability to explain server-side concepts including routing, sessions, authentication, and middleware.	25%
Correct implementation of protected routes, session persistence, login/logout functionality, and application testing.	20%
Code organization, readability, modularity, documentation, and adherence to server-side development best practices.	15%

8	**Aim:** To develop a database-driven web application that performs CRUD (Create, Read, Update, Delete) operations using server-side technologies while ensuring proper input validation and data sanitization**.**	CO5

Industry Scenario

TechMart Electronics is replacing its manual inventory management system with a web-based application to efficiently manage product information. As a Full-Stack Web Developer, you have been assigned the responsibility of developing the product management module for the application.

The management requires the system to allow authorized users to add, view, update, and delete product records from the database. To maintain data integrity and application security, all user inputs must be validated and sanitized before being stored in the database. The application should provide meaningful feedback for successful operations and appropriate error messages for invalid inputs or unsuccessful transactions.

Based on the above requirements, develop a database-connected web application that performs CRUD operations using appropriate server-side technologies and database connectivity.

Tasks to be Performed

1. Create a database and appropriate table(s) for storing product information.
2. Configure the server-side application to establish a connection with the database.
3. Design a product entry form to capture details such as Product ID, Product Name, Category, Price, Quantity, and Description.
4. Implement the Create operation to insert validated product records into the database.
5. Implement the Read operation to retrieve and display all product records in a tabular format.
6. Implement the Update operation to modify existing product information.
7. Implement the Delete operation with a confirmation mechanism before removing records.
8. Apply server-side validation and input sanitization to prevent invalid or malicious data from being stored.
9. Display appropriate success and error messages for each CRUD operation.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Explain the importance of CRUD operations in database-driven web applications. Provide suitable examples from your implementation.
2. Differentiate between client-side validation and server-side validation. Why should both be implemented in modern web applications?
3. Discuss the importance of input sanitization before storing user data in a database. What security risks may arise if sanitization is not performed?

Supplementary Problems

1. Develop a Student Information Management System that performs CRUD operations on student records with appropriate validation.
2. Design an Employee Management System that allows administrators to manage employee information using database connectivity.

3. Create a Library Book Management System that enables librarians to add, search, update, and remove book records securely.

Key Skills to be Addressed

- Establishing database connectivity.
- Performing Create, Read, Update, and Delete (CRUD) operations.
- Implementing server-side validation.
- Applying input sanitization techniques.
- Managing relational database records.
- Handling database errors and exceptions.
- Developing secure database-driven web applications.

Learning Outcome

Upon successful completion of this practical, students will be able to develop secure database-driven web applications that perform CRUD operations using server-side technologies while ensuring proper validation, sanitization, and data integrity.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Node.js
- Express.js
- MySQL / MariaDB
- MySQL Workbench or phpMyAdmin
- Google Chrome / Mozilla Firefox
- Browser Developer Tools

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design and Implementation
- **0.5 Hour:** Testing, Debugging, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of database connectivity and CRUD operations with proper data handling.	40%
Ability to explain database interaction, CRUD concepts, validation, and sanitization techniques.	25%
Correct handling of database operations, error conditions, and data validation with meaningful user feedback.	20%
Code organization, database design, naming conventions, documentation, and adherence to secure coding practices.	15%

9	**Aim:** To develop a web application that integrates REST APIs and exchanges data in JSON format for retrieving, processing, and displaying dynamic information. **Industry Scenario** QuickShop Online Marketplace is enhancing its e-commerce platform by integrating third-party services to provide customers with real-time product	CO5

information. As a Full-Stack Web Developer, you have been assigned the task of integrating a REST API into the web application to retrieve product data dynamically and present it in a user-friendly format.
The management expects the application to communicate with RESTful web services, process JSON responses, display the retrieved information dynamically, and handle communication errors effectively. The application should demonstrate the ability to consume API data and exchange information using standard HTTP methods while maintaining a responsive user experience. Based on the above requirements, develop a web application that integrates a REST API and demonstrates JSON-based data exchange.

Tasks to be Performed

1. Create a web application capable of communicating with a REST API.
2. Integrate a public REST API or a locally developed REST service into the application.
3. Send HTTP requests using appropriate methods such as GET and POST.
4. Retrieve and parse JSON data received from the API.
5. Display the retrieved information dynamically in a structured format such as cards or tables.
6. Send JSON data to the server through an API request wherever applicable.
7. Implement appropriate error handling for failed API requests and invalid responses.
8. Display meaningful success and error messages based on API responses.
9. Test the application using different API endpoints and verify the correctness of the exchanged data.
10. Document the API endpoints, request methods, and JSON structure used in the application.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Explain the architecture of a REST API. How does it enable communication between client-side and server-side applications?
2. Discuss the advantages of using JSON over XML for data exchange in modern web applications.
3. Differentiate between the HTTP methods GET, POST, PUT, and DELETE. Identify suitable scenarios for each method.
4. While consuming a REST API, the server returns an error response. Explain how your application should detect, interpret, and handle different HTTP status codes.

Supplementary Problems

1. Develop a Weather Information Portal that retrieves weather details from a public REST API and displays the information dynamically.
2. Create a Book Search Application that fetches book details from an online API based on user search criteria.
3. Design a Movie Information System that retrieves movie details from a public API and displays posters, ratings, and descriptions using JSON data.

Key Skills to be Addressed

- Understanding RESTful web services.
- Integrating REST APIs into web applications.
- Sending and receiving HTTP requests.
- Processing and parsing JSON data.
- Implementing API error handling.
- Dynamically updating webpage content using API responses.
- Developing API-driven web applications.

Learning Outcome

Upon successful completion of this practical, students will be able to integrate REST APIs into web applications, exchange data using JSON, process API responses, and develop dynamic, data-driven web applications using modern web technologies.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Node.js
- Express.js
- HTML5
- CSS3
- JavaScript (ES6)
- Google Chrome / Mozilla Firefox
- Browser Developer Tools
- Postman (for API testing)
- Public REST API or Local REST API

Total Hours of Engagement

2 Hours

- **1.5 Hours:** Design and Implementation
- **0.5 Hour:** API Testing, Debugging, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of REST API integration, HTTP request handling, JSON parsing, and dynamic data presentation.	**40%**
Ability to explain REST architecture, HTTP methods, JSON data exchange, and API communication concepts.	**25%**
Proper handling of API responses, error conditions, HTTP status codes, and user feedback mechanisms.	**20%**
Code organization, readability, API documentation, naming conventions, and adherence to web development best practices.	**15%**

10	**Aim:** To understand common web security vulnerabilities and implement secure coding techniques, password protection mechanisms, and defensive programming practices in web applications. **Industry Scenario**	CO5

FinSecure Solutions Pvt. Ltd. is conducting a security review of its employee management portal before deploying it for organizational use. As a Web Security Analyst, you have been assigned to identify common security weaknesses in the application and recommend appropriate defensive measures.
The management requires a controlled demonstration of common web vulnerabilities such as Cross-Site Scripting (XSS) and SQL Injection to understand how insecure coding practices can compromise application security. The objective is not to exploit systems but to analyze potential risks and implement secure coding techniques, proper input validation, parameterized database queries, secure password handling, and other defensive mechanisms to enhance the overall security of the application.
Based on the above requirements, analyze the application for common security vulnerabilities and implement appropriate defensive coding practices.

Tasks to be Performed

- Develop a simple web application containing user input fields for login or data entry.
- Demonstrate how improper input validation can lead to Cross-Site Scripting (XSS) in a controlled environment.
- Implement appropriate input validation and output encoding techniques to prevent XSS attacks.
- Explain how SQL Injection vulnerabilities occur and demonstrate secure database access using parameterized queries (prepared statements).
- Implement strong password policies including minimum length, complexity requirements, and confirmation validation.
- Demonstrate secure password storage using appropriate password hashing techniques provided by the selected server-side framework.
- Implement appropriate error handling to avoid exposing sensitive system or database information to end users.
- Review the application and document the security measures implemented to improve its overall security.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

- Explain how Cross-Site Scripting (XSS) affects the confidentiality and integrity of web applications. How can secure coding practices prevent such attacks?
- Compare dynamic SQL queries with parameterized queries (prepared statements). Why are parameterized queries recommended for preventing SQL Injection?
- Discuss the importance of password hashing and secure password policies. Why should passwords never be stored in plain text?

Supplementary Problems

- Analyze a simple login module and identify potential security weaknesses. Recommend appropriate countermeasures.
- Enhance an existing registration form by implementing secure password validation, hashing, and proper error handling.

Key Skills to be Addressed

<table>
<tr>
<td></td>
<td>

Understanding common web security vulnerabilities.
Applying secure coding practices.
Implementing input validation and output encoding.
Using parameterized database queries.
Implementing secure password management.
Identifying common security risks in web applications.
Developing security-aware web applications.

Learning Outcome
Upon successful completion of this practical, students will be able to identify common web security vulnerabilities, explain their impact, and implement secure coding practices including input validation, parameterized queries, secure password handling, and defensive programming techniques to improve application security.

Software / Tools / Platforms to be Used

Visual Studio Code
Node.js
Express.js
MySQL / MariaDB
HTML5
CSS3
JavaScript (ES6)
Google Chrome / Mozilla Firefox
Browser Developer Tools

Total Hours of Engagement
2 Hours

1.5 Hours: Implementation and Security Analysis
0.5 Hour: Testing, Discussion, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)
<table>
<tr><th>Criteria</th><th>Weight (%)</th></tr>
<tr><td>Correct identification of common web vulnerabilities and implementation of secure coding practices.</td><td>40%</td></tr>
<tr><td>Ability to explain XSS, SQL Injection awareness, password security, and defensive programming concepts.</td><td>25%</td></tr>
<tr><td>Proper implementation of input validation, parameterized queries, secure password handling, and error management.</td><td>20%</td></tr>
<tr><td>Code quality, documentation, adherence to secure coding standards, and overall application organization.</td><td>15%</td></tr>
</table>
</td>
<td></td>
</tr>
<tr>
<td>11</td>
<td>
Aim: To develop a responsive website using Bootstrap and Tailwind CSS frameworks and evaluate its compatibility across multiple browsers and devices using browser developer tools.

Industry Scenario
FoodExpress Pvt. Ltd. is launching a responsive online food ordering platform to provide customers with a seamless browsing experience across desktops, tablets, and smartphones. As a Front-End UI Developer, you have been assigned
</td>
<td>CO2, CO4, CO5, CO6</td>
</tr>
</table>

the responsibility of designing the customer-facing website using modern CSS frameworks.
The management requires the website to be responsive, visually appealing, and compatible with different browsers and screen sizes. The application should utilize reusable UI components, responsive layouts, and utility classes provided by Bootstrap and Tailwind CSS. Before deployment, the website must be tested using browser developer tools to ensure consistent appearance and functionality across multiple devices and browsers.
Based on the above requirements, develop a responsive website using Bootstrap and Tailwind CSS and evaluate its compatibility across different browsers and devices.

Tasks to be Performed

1. Create a responsive website for an online food ordering system using HTML5.
2. Integrate Bootstrap into the project and utilize its grid system, navigation bar, cards, buttons, forms, and other UI components.
3. Apply Tailwind CSS utility classes to design responsive layouts and customize the appearance of webpage elements.
4. Develop a responsive homepage containing a navigation menu, food categories, featured items, promotional banners, and contact information.
5. Ensure the website adapts appropriately to desktop, tablet, and mobile screen resolutions using responsive framework features.
6. Test the website using browser developer tools by simulating different devices and screen sizes.
7. Verify browser compatibility by executing the application in at least two modern web browsers.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Compare Bootstrap and Tailwind CSS with respect to design philosophy, customization, development speed, and maintainability. Which framework would you recommend for large-scale web applications? Justify your answer.
2. Explain the importance of cross-browser compatibility in web application development. What factors should be considered while testing websites on different browsers and devices?
3. Discuss how browser developer tools assist developers in debugging responsive layouts, inspecting elements, and identifying rendering issues.

Supplementary Problems

1. Develop a responsive College Event Management Website using Bootstrap and Tailwind CSS.
2. Create a responsive Hotel Booking Landing Page with navigation, room listings, pricing, testimonials, and contact information.
3. Design a responsive Personal Portfolio Website and evaluate its compatibility across multiple browsers and screen resolutions.

Key Skills to be Addressed

- Developing responsive websites using Bootstrap.
- Designing modern user interfaces using Tailwind CSS.
- Applying responsive layout techniques.
- Using browser developer tools for debugging and inspection.
- Performing cross-browser compatibility testing.
- Identifying and resolving browser-specific rendering issues.
- Comparing modern front-end frameworks for web development.

Learning Outcome

Upon successful completion of this practical, students will be able to develop responsive websites using Bootstrap and Tailwind CSS, evaluate browser compatibility across multiple devices, utilize browser developer tools for debugging, and select appropriate front-end frameworks for modern web application development.

Software / Tools / Platforms to be Used

- Visual Studio Code
- HTML5
- CSS3
- Bootstrap 5
- Tailwind CSS
- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Browser Developer Tools

Total Hours of Engagement

3 Hours

- **2.5 Hours:** Design, Implementation, and Responsive Testing
- **0.5 Hour:** Browser Compatibility Testing and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of responsive layouts using Bootstrap and Tailwind CSS, including appropriate use of framework components and utility classes.	**40%**
Ability to explain responsive web design concepts, Bootstrap, Tailwind CSS, browser compatibility, and responsive testing techniques.	**25%**
Effective use of browser developer tools to identify and resolve responsiveness and cross-browser compatibility issues.	**20%**
Code organization, readability, framework usage, documentation, and adherence to responsive web development best practices.	**15%**

12	**Aim:** To design, develop, test, and deploy a complete responsive web application by integrating front-end, server-side, database, security, and responsive web technologies using industry-standard development practices. **Project Statement**	CO1, CO2, CO3, CO4, CO5,

Design and develop a complete responsive web application based on a real-world problem of your choice. The project should integrate the concepts learned throughout the course, including HTML5, CSS3, JavaScript, server-side processing, database connectivity, REST API integration, responsive design frameworks, validation, debugging, secure coding practices, and version control using Git and GitHub.
Students may select application domains such as education, healthcare, banking, e-commerce, tourism, inventory management, event management, library management, agriculture, or any other relevant real-world problem approved by the course instructor.

CO6

Total Hours of Engagement
4 Hours

- **3.0 Hours:** Design, Development, Integration, and Testing
- **1.0 Hour:** Project Demonstration, Viva, Documentation Review, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Successful integration of front-end, server-side processing, database connectivity, REST API, validation, security, and responsive design into a complete web application.	**40%**
Ability to explain the project architecture, implementation methodology, technology stack, and design decisions.	**25%**
Functionality, responsiveness, security, testing, debugging, documentation, and effective use of Git/GitHub.	**20%**
Code quality, modular design, project organization, presentation, and adherence to industry-standard web development practices.	**15%**